

Traccia A

Esercizio 1 (A)

Si considerino le seguenti funzioni che implementano un Albero Binario in cui la parte informativa di ogni nodo è un numero intero:

- *createTree()* che restituisce un albero vuoto,
- *insert(A, x)* che inserisce il nodo x nell'albero A ,
- *left(A)* che restituisce il sottoalbero sinistro di A ,
- *right(A)* che restituisce il sottoalbero destro di A ,
- *info(A)* che restituisce il valore associato al nodo radice di A ,
- *setInfo(A)* che modifica il valore associato alla radice di A ,
- *isNull(A)* restituisce *True* se l'albero A è vuoto e *False* altrimenti.

Si realizzi una efficiente funzione *checkTree(A, z1, z2)* che dato un Albero binario A , e due interi $z1$ e $z2$ con $z1 < z2$, restituisce *True* se e solo se esiste in A un sottoalbero B il cui nodo radice non sia un nodo foglia di A e tale che per ogni nodo foglia q di *right(B)* (il sottoalbero destro di B), valgano entrambe le seguenti proprietà:

- $info(B) + info(q) \geq z1$,
 - $info(B) + info(q) \leq z2$.
- $\Rightarrow$ $z1 \leq info(B) + info(q) \leq z2$

Si discuta, motivandola adeguatamente, la complessità temporale e spaziale del metodo nei casi migliore e peggiore, esplicitando quali configurazioni costituiscano rispettivamente il caso migliore e il caso peggiore per entrambe le complessità.

Esercizio 2 (A)

Si considerino le seguenti funzioni che implementano un grafo non orientato:

- *createGraph(m)*, che, dato un valore intero m in input, crea un grafo con m nodi identificati da numeri interi compresi in $[0, m-1]$;
- *insertEdge(G, x, y)*, che, dato un grafo G e una coppia di nodi x e y , aggiunge un arco dal nodo x al nodo y ;
- *deleteEdge (G, x, y)*, che, dato un grafo G e una coppia di nodi x e y , rimuove, se presente, l'arco dal nodo x al nodo y ;
- *deleteNode(G,x)*, che, dato un grafo G e un nodo x , la funzione rimuove il nodo x da G . Se il nodo x possiede archi incidenti (entranti o uscenti), oppure x non è presente nel grafo G , il metodo solleva un'eccezione.
- *existsEdge (G, x, y)*, che, dato un grafo G e una coppia di nodi x e y , restituisce *True* se esiste in G un arco dal nodo x al nodo y , *False* altrimenti;
- *size(G)*, che, dato un grafo G restituisce il numero di nodi del grafo;
- *nodes(G)*, che, dato un grafo G restituisce una lista contenente gli identificatori dei nodi di G .

Scrivere una funzione **independent(G,S)**, che prende in input un grafo non orientato $G = (V, E)$, con $V = \{0, \dots, n-1\}$, e una lista di nodi S . La funzione deve restituire *True* se e solo se sono verificate tutte le seguenti condizioni:

- **S non banale.** Il sottoinsieme S è non banale, ovvero non è vuoto e non coincide con l'intero insieme dei nodi.
- **S indipendente.** Gli elementi in S corrispondono a nodi in G che non hanno archi tra loro. Formalmente, verificare che valga la seguente condizione $\forall x, y \in S \subset V, x \neq y \Rightarrow \neg \text{existsEdge}(G,x,y)$.
- **Valutazione del residuo.** Rimuovendo da G tutti i nodi in S e gli archi incidenti si ottiene un sottografo $G'=(V',E')$ con $V'=V \setminus S$, che è un albero binario di ricerca.

Si noti che se una qualunque delle condizioni fallisce, la funzione *independent* dovrà restituire *False*.

Si consideri inoltre che, per per quanto riguarda il punto 3, per manipolare l'albero è possibile utilizzare le primitive fornite nell'Esercizio 1 della traccia.

Si analizzi e si motivi la complessità temporale nel caso peggiore.

Traccia B

Esercizio 1 (B)

Si considerino le seguenti funzioni che implementano un Albero Binario in cui la parte informativa di ogni nodo è un numero intero:

- *createTree()* che restituisce un albero vuoto,
- *insert(A, x)* che inserisce il nodo x nell'albero A ,
- *left(A)* che restituisce il sottoalbero sinistro di A ,
- *right(A)* che restituisce il sottoalbero destro di A ,
- *info(A)* che restituisce il valore associato al nodo radice di A ,
- *setInfo(A)* che modifica il valore associato alla radice di A ,
- *isNull(A)* restituisce *True* se l'albero A è vuoto e *False* altrimenti.

Si realizzi una efficiente funzione *checkTreeAbs(A, z1, z2)* che dato un Albero binario A , e due interi $z1$ e $z2$ con $z1 < z2$, restituisce *True* se e solo se esiste in A un sottoalbero B il cui nodo radice non sia un nodo foglia di A e tale che per ogni nodo foglia q di B valgano entrambe le seguenti proprietà:

- $z1 \leq \text{abs}(\text{info}(q) - \text{info}(B)) \leq z2$,
- la somma dei nodi lungo il cammino che va dalla radice di B alla foglia q (esclusa) è minore o uguale a $2 \cdot (z1 + z2)$.

dove *abs()* è il simbolo di valore assoluto.

Si discuta, motivandola adeguatamente, la complessità temporale e spaziale del metodo nei casi migliore e peggiore, esplicitando quali configurazioni costituiscano rispettivamente il caso migliore e il caso peggiore per entrambe le complessità.

Esercizio 2 (B)

Si considerino le seguenti funzioni che implementano un grafo non orientato:

- *createGraph(m)*, che, dato un valore intero m in input, crea un grafo con m nodi identificati da numeri interi compresi in $[0, m-1]$;
- *insertEdge(G, x, y)*, che, dato un grafo G e una coppia di nodi x e y , aggiunge un arco dal nodo x al nodo y ;
- *deleteEdge (G, x, y)*, che, dato un grafo G e una coppia di nodi x e y , rimuove, se presente, l'arco dal nodo x al nodo y ;
- *deleteNode(G,x)*, che, dato un grafo G e un nodo x , la funzione rimuove il nodo x da G . Se il nodo x possiede archi incidenti (entranti o uscenti), il metodo solleva un'eccezione.
- *existsEdge (G, x, y)*, che, dato un grafo G e una coppia di nodi x e y , restituisce True se esiste in G un arco dal nodo x al nodo y , False altrimenti;
- *size(G)*, che, dato un grafo G restituisce il numero di nodi del grafo;
- *nodes(G)*, che, dato un grafo G restituisce una lista contenente gli identificatori dei nodi di G .

Scrivere una funzione **clique(G,S)**, che prende in input un grafo non orientato $G = (V,E)$, con $V = \{0,...,n-1\}$, e una lista di nodi S . La funzione deve restituire True se e solo se sono verificate tutte le seguenti condizioni:

1. **S non banale.** Il sottoinsieme S è non banale, ovvero non è vuoto e non coincide con l'intero insieme dei nodi.
2. **S clique.** Gli elementi in S corrispondono a nodi in G che non hanno archi tra loro in G , ovvero verificare la seguente condizione
 $\forall x, y \in S \subset V, x \neq y \Rightarrow \text{existsEdge}(G,x,y)$.
3. **Valutazione del residuo.** Rimuovendo da G tutti i nodi in S e gli archi incidenti si ottiene un sottografo $G'=(V',E')$ con $V'=V \setminus S$, che è un albero binario di ricerca.

Si noti che se una qualunque delle condizioni fallisce, la funzione *clique* dovrà restituire *False*. Inoltre, per quanto riguarda il punto 3, per manipolare l'albero è possibile utilizzare le primitive fornite nell'esercizio 1 della traccia.

Si analizzi e si motivi la complessità temporale nel caso peggiore.